



GE VERNOVA

ADMS

16.25.1

Permit Builder User Guide

Copyright

Copyright 2026, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Change History

Date	Change Description
Jan 26, 2023	ADMS 3.16: • Editorial and formatting updates.
Oct 20, 2023	ADMS 16.1.0: • The product version numbering has changed. The prefix '3' is no longer included in the version numbering for ADMS.
Apr 16, 2024	ADMS 16.7.0: • Updated document format and styling.

Contents

1. Introduction	5
1.1. Terminology	6
2. UI Components of Permit Builder	8
2.1. Tree Nodes	8
2.2. Three-State Check Boxes	8
2.3. Permission Modes	9
2.4. Permission Controls	9
2.5. Node Text	10
2.6. Entity Type	10
2.7. Status Bar	11
3. Basic Permit Builder Operations	12
3.1. Operations with Nodes	12
3.1.1. Adding a Node	12
3.1.2. Editing a Node	13
3.1.3. Removing a Node	14
3.1.4. Copying a Node	15
3.1.5. Cutting a Node	15
3.1.6. Pasting a Node	15
3.1.7. Finding a Node	16
3.1.8. Expanding and Collapsing a Subtree	17
3.2. Operations with User Types and Workstation Types	18
3.2.1. Adding a User Type or Workstation Type	18
3.2.2. Renaming a User Type or Workstation Type	19
3.2.3. Deleting a User Type or Workstation Type	20
3.3. Modifying Access Levels for AORs	20
3.4. Saving and Importing the Model	20
3.4.1. Validation Errors	21
4. Building the AORRef Hierarchy	22
4.1. Building a Sample Tree	22
4.2. The AOR Reference XML Schema	23
4.3. Building a Basic Hierarchy	26
4.4. Building the Hierarchy via an External Script	26
4.5. Building a User Type and Workstation Type Hierarchy	27
5. Validating the AORRef Hierarchy	28
5.1. Validating the Model	28
5.2. Defining the Validation Criteria	29
5.2.1. Circular References	29
5.2.2. Invalid AOR Name	30
5.2.3. Inconsistency between the AOR Name and IsAOR Attributes	30
6. Creating Hdbimport Files	31

6.1. Exporting the Model in Hdbimport Format	31
6.2. Example of the Hdbimport File.....	32
6.3. Exporting an Intersection Model File.....	32
6.4. Exporting Models from the Command Line	33
7. Modifying the Permit Model with a Script	36
7.1. Running the Script	37
7.2. Changing the Workstation Type Model.....	38
7.3. Verifying Script Results	39

1. Introduction

The Permit Builder is designed for creating a hierarchical model of Areas of Responsibility (AORs) for a Permit model. It can also be used to create hierarchical models for User Types and Workstation Types based on the basic AOR model.

As an output, the Permit Builder produces Hdbimport files that contain AREA and HMODE records for the PERMITNV database. In addition, it can produce an intersection model file with the permission models for User Types and Workstation Types to be used to build an Intersection model and modify the Permit database.

Note

After you import an Hdbimport file into the PERMITNV database, set HACTIVE_ITEMS to True in the PERMITNV database using HDBRIO. Otherwise, the hierarchical AOR model will not appear in the Permission Management plug-in.

For information about the PERMIT application and the PERMITNV database, refer to the *PERMIT User Guide*, which is distributed with the Habitat product.

The below image shows the Permit Builder main window, with the tree structure and the permission mode options on the left side, the Configuration pane on the right side, and the status bar across the bottom. For more information about the Permit Builder user interface, see [UI Components of Permit Builder](#).



Figure 1. Permit Builder

1.1. Terminology

The following terms and acronyms are used throughout this document:

- **Area:** An operational environment that can be given permissions. The environment can be deemed critical if it must be under the control of at least one active workstation.
- **Area of Responsibility (AOR):** An Area of Responsibility represents an entity that can be assigned to a user. For an electrical system, a device is typically assigned to an AOR, and the AOR is assigned to a user.

An AOR can also represent functionality that a user may or may not be allowed to perform. In this case, a software function is assigned to an AOR, and the AOR in turn is assigned to a user.

The concept of an AOR generally matches the current usage of an AREA record in the Habitat PERMIT database. These terms are used interchangeably within this document.

- **Area of Responsibility Reference (AORRef):** The user interface(UI) displays user permissions in a hierarchical, tree view format. Each node in the hierarchy is an AORRef. An AORRef can serve either of two purposes, which are not mutually exclusive:
 - An AORRef can point to or "reference" an AOR. This is the primary means for binding how the AORs are displayed on the UI to the underlying AREA records in PERMIT.

- An AORRef can contain other AORRefs, allowing for the hierarchical organization of AORs in a tree view format.

AORRefs are modeled as HMODE records in the PERMITNV database.

- **Functional AOR:** A functional AOR is an AOR that relates to functional capability, as opposed to being tied to specific equipment. For example, the AUDTEMP functional AOR controls the user's ability to disable audible alarms temporarily.
- **Internal AOR:** As described above, AORRefs can be containers for other AORRefs, as well as pointers to AREA records in PERMIT. If an AORRef plays both roles, the AORRef is modeled as two distinct HMODE records in PERMIT. The first HMODE represents the container. The second HMODE exists as a child of the container; it points to an AREA record in PERMIT, and this AREA record is termed an "internal AOR".

By modeling internal AORs in this way, all AREA records exist as end-nodes or "leaves" of the tree. From a server perspective, this has many advantages. However, the UI can be configured to show or hide internal AORs.

- **HMODE:** This term is used, in a broad sense, to designate area permissions modeled and managed hierarchically (hence the "H") from a Browser Permission plug-in initially developed for ADMS. Instead of managing permissions through predefined modes, the Permission plug-in allows management at the area level or group of areas (a hierarchy) level; however, the term "MODE" has been kept. Permissions for applications are always managed through MODEs.
- **Mode:** A named set of permissions to the applications and areas of the control system.
- **User:** An identification (i.e., a user ID) that relates to a human user of the control system. This includes administrators as well as operators.
- **User Type:** A classification of users based on the modes they use.
- **Workstation:** A physical component used to access the control system. A workstation can be defined to cover critical areas.
- **Workstation Type:** A classification of workstations based on the modes they use.

2. UI Components of Permit Builder

This chapter describes the various components of the Permit Builder user interface.

2.1. Tree Nodes

The main window consists of a collection of nodes, arranged in a tree-like structure. Each row in the tree corresponds to a single AORRef.

In the below image, the AORRef labeled "Baker" is circled in red.

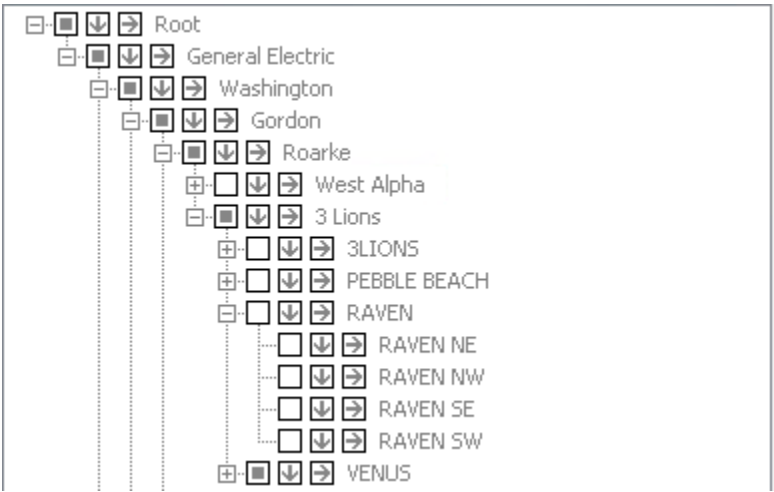


Figure 2. "RAVEN NE" AORRef Label

2.2. Three-State Check Boxes

Each tree node consists of a series of boxes followed by a text label. The first of these boxes is a three-state check box.

The three possible check box states are: checked, unchecked, and grayed. The check box state indicates the level of permissions assigned to the given area and its descendants. These states are summarized in the below table.

Note

The three-state check boxes are only visual indicators. Selecting a checkbox has no effect on the display.

Table 1. Check Box States

Image	State	Description
☒	Checked	Indicates that the area and all descendants have full access.

Image	State	Description
☐	Unchecked	Indicates that neither the area nor any of its descendants have full access.
☐	Grayed	Indicates that the area is neither checked nor unchecked (i.e., one or more areas have full access, but not all areas have full access).

2.3. Permission Modes

The permission mode determines which permission controls are visible and configurable on the UI. For example, if you select the Read Access option, the Read Down and ReadRight permission controls appear.

The permission mode options are shown in the bottom-left corner of the Permit Builder.

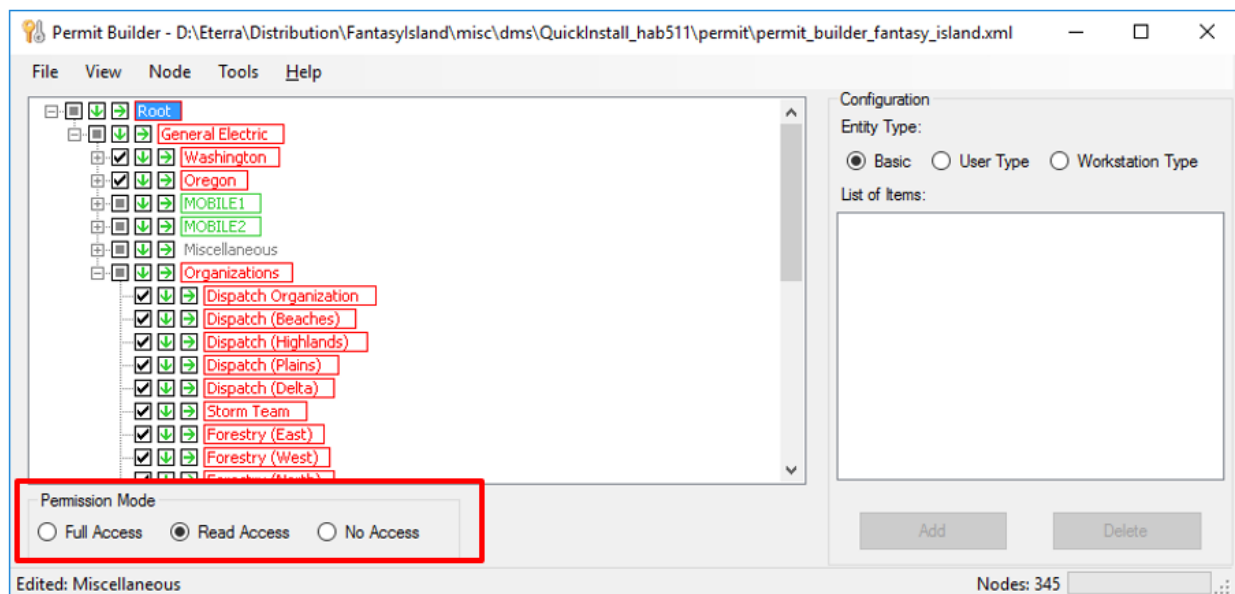


Figure 3. Permission Mode List

2.4. Permission Controls

Each tree node contains one or more permission controls, which appear to the right of a check box. These controls are the primary mechanism for changing a user's permissions. For example, to request full access for a given AORRef and all of its descendants, select Full Access from the Permission Mode list (see [Permission Mode](#)), and click on the Full Down image (red down arrow). The below table summarizes the behavior of these controls.

Table 2. Permission Controls

Image	State	Description
	Full Down	Requests full access for the selected area and all descendants.
	Full Right	Requests full access for only the selected area.

Image	State	Description
	Read Down	Requests read access for the selected area and all descendants.
	Read Right	Requests read access for only the selected area.
	None Down	Requests no access for the selected area and all descendants.
	None Right	Requests no access for only the selected area.

2.5. Node Text

The text color and the background color of the node text indicate the user permissions for a given area. The below table describes these color indicators.

Table 3. Node Text Colors

Image	State	Description
	Full	The user has full access permission for the area.
	Read	The user has read access permission for the area.
Gray Text	No Access	The Access mode for the area is NoAccess (i.e., the user never gains access to the area).

2.6. Entity Type

On the Permit Builder main window, the Configuration pane includes options for selecting, adding, renaming, and deleting User Type and Workstation Types.

Note

Only one basic entity type (AOR model) can be defined. You can neither change its name nor add more basic models.

The left pane shows the tree-like structure for the currently selected entity type. For example, if you select the User Type option (as shown in the below image), the UI presents the structure for the currently selected User Type.

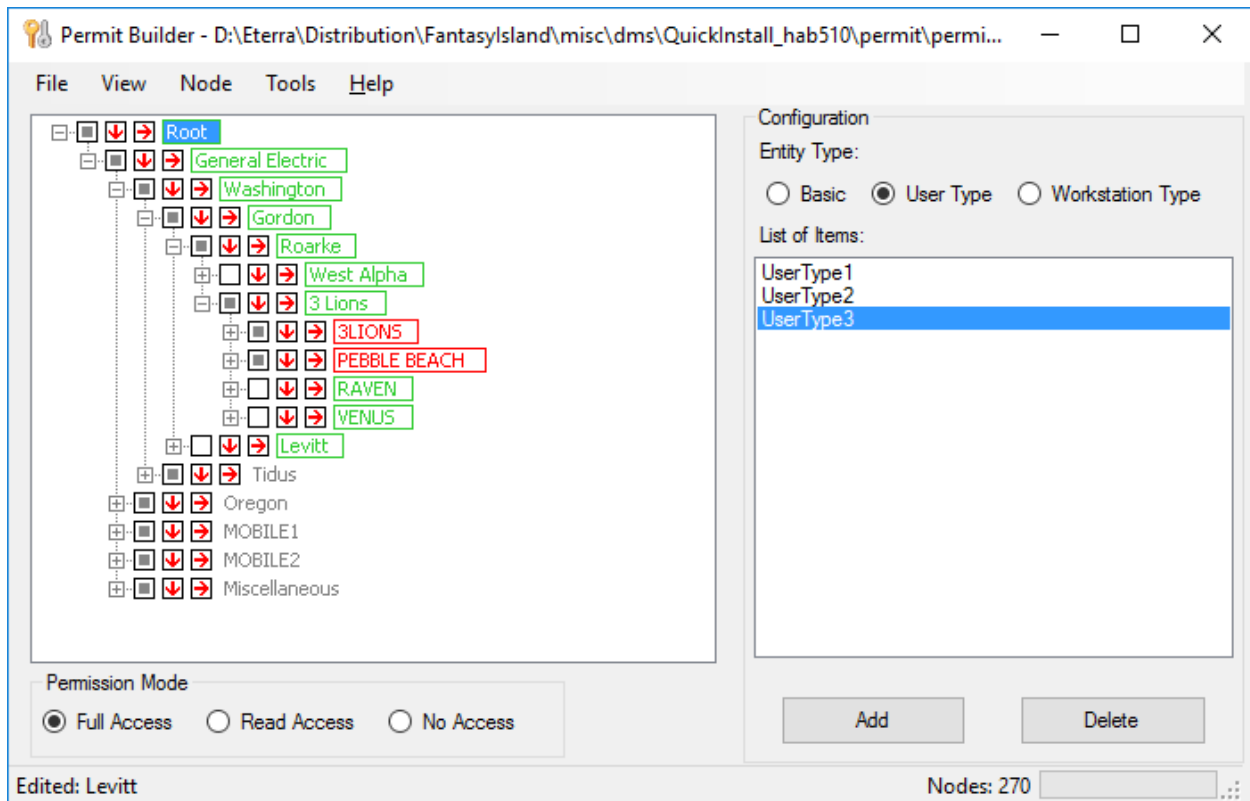


Figure 4. Viewing the Structure for the User Type Entity

2.7. Status Bar

The status bar is located at the bottom of the Permit Builder. The last user action appears at the left of the status bar. The total number of nodes in the model appears at the right of the status bar.

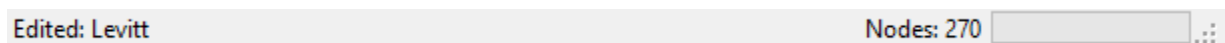


Figure 5. Status Bar

3. Basic Permit Builder Operations

This chapter describes the basic operations of the Permit Builder.

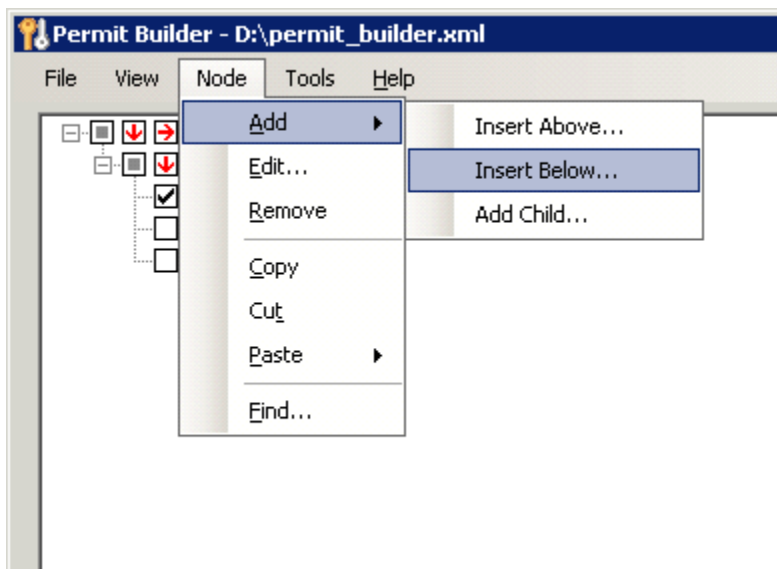
3.1. Operations with Nodes

This section lists operations with AOR nodes.

3.1.1. Adding a Node

To add an AORRef node to the hierarchy:

1. Select a node.
2. On the Node menu, point to Add, and then select one of the following options:
 - **Insert Above:** Insert a (sibling) node above the selected node.
 - **Insert Below:** Insert a (sibling) node below the selected node.
 - **Add Child:** Add a child to the selected node.

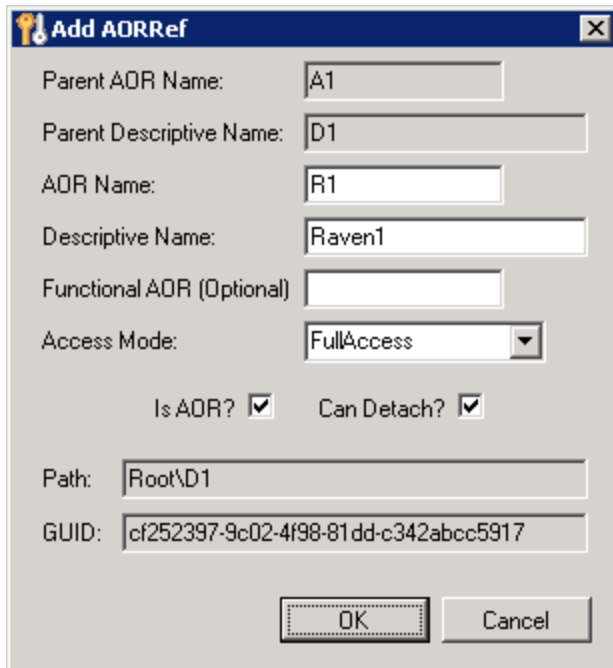


The first two options add a node on the same level as the selected node. The Add Child option adds a node as a child to the selected node.

i Note

You can also add a node from the shortcut menu. This is true for most of the basic operations described in this chapter.

After you select one of the three options above, the AddAORRef dialog box appears.

The image shows a Windows-style dialog box titled "Add AORRef". It contains several input fields and checkboxes. The fields are: "Parent AOR Name:" with value "A1", "Parent Descriptive Name:" with value "D1", "AOR Name:" with value "R1", "Descriptive Name:" with value "Raven1", "Functional AOR (Optional)" which is empty, and "Access Mode:" with a dropdown menu showing "FullAccess". Below these are two checkboxes: "Is AOR?" and "Can Detach?", both of which are checked. At the bottom, there are two more fields: "Path:" with value "Root\D1" and "GUID:" with value "cf252397-9c02-4f98-81dd-c342abcc5917". At the very bottom are "OK" and "Cancel" buttons.

Add AORRef

Parent AOR Name: A1

Parent Descriptive Name: D1

AOR Name: R1

Descriptive Name: Raven1

Functional AOR (Optional)

Access Mode: FullAccess

Is AOR? ☒ Can Detach? ☒

Path: Root\D1

GUID: cf252397-9c02-4f98-81dd-c342abcc5917

OK Cancel

3. Complete the dialog box and click OK.

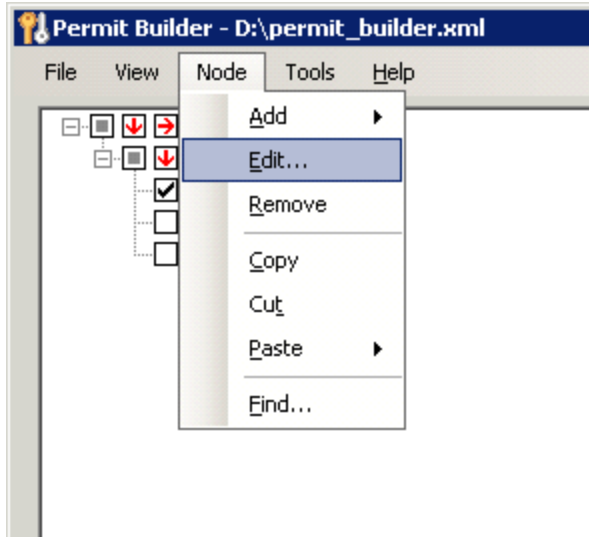
Note

The AOR Name field is required only if the IsAOR? check box is selected.

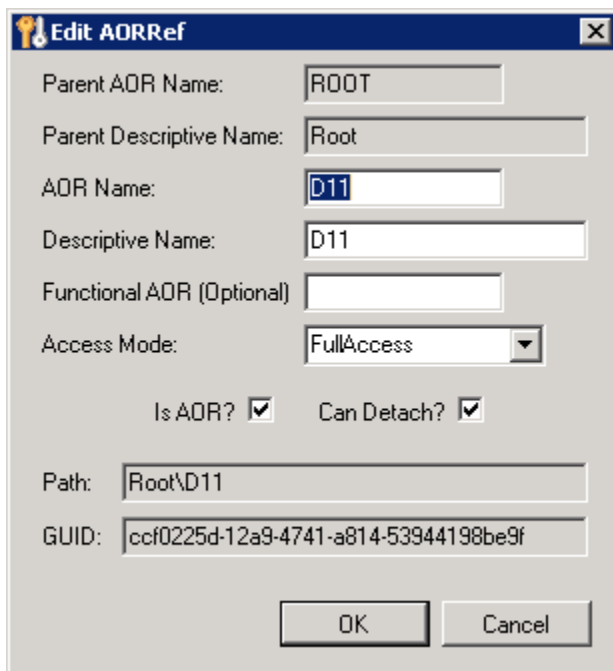
3.1.2. Editing a Node

To edit an AORRef node in the hierarchy:

1. Select a node.
2. On the Node menu, click Edit.



3. In the Edit AORRef dialog box, make any necessary changes and click OK.



3.1.3. Removing a Node



Warning

Removing an AORRef is permanent. There is no built-in "undo" functionality.

To remove an AORRef node from the hierarchy:

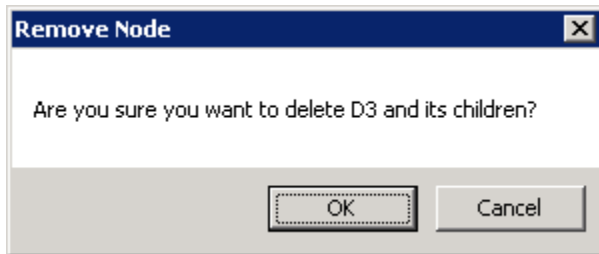
1. Select a node.

2. On the Node menu, click Remove.

A confirmation message appears.

3. To delete the AORRef (and all of its children, if any), click OK.

To cancel the operation, click Cancel.



3.1.4. Copying a Node

After you copy an AORRef, the node remains on the clipboard until it is overwritten by a subsequent copy or cut operation. The clipboard is never cleared.

To copy an AORRef node from the hierarchy:

1. Select a node.
2. On the Node menu, click Copy.

3.1.5. Cutting a Node

After you cut an AORRef, the node remains on the clipboard until it is overwritten by a subsequent copy or cut operation. The clipboard is never cleared.

To cut an AORRef node from the hierarchy:

1. Select a node.
2. On the Node menu, click Cut.

3.1.6. Pasting a Node

Note

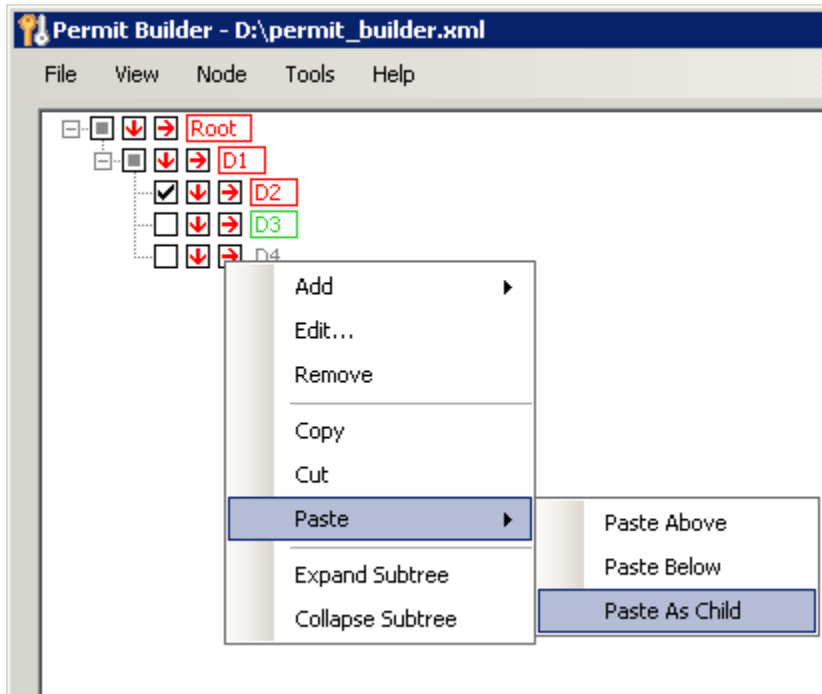
In order to paste a node into the hierarchy, there must be an AORRef already on the clipboard from a copy or cut operation.

To paste an AORRef node to the hierarchy:

1. Select a node.

2. On the Node menu, choose Paste, then select one of the following options:

- **Paste Above:** Paste a (sibling) node above the selected node.
- **Paste Below:** Paste a (sibling) node below the selected node.
- **Paste As Child:** Paste a child into the selected node.



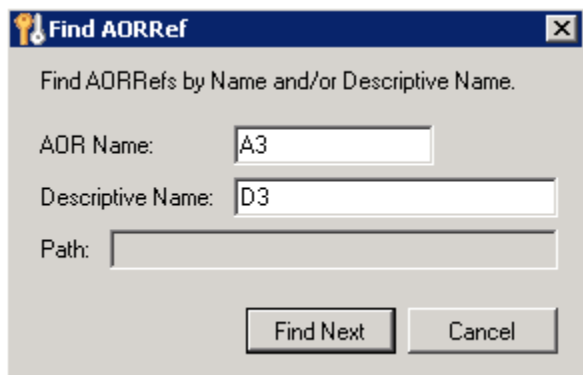
The first two options paste a node on the same level as the selected node. The Paste As Child option pastes a node as a child to the selected node.

3.1.7. Finding a Node

To find an AORRef node in the hierarchy:

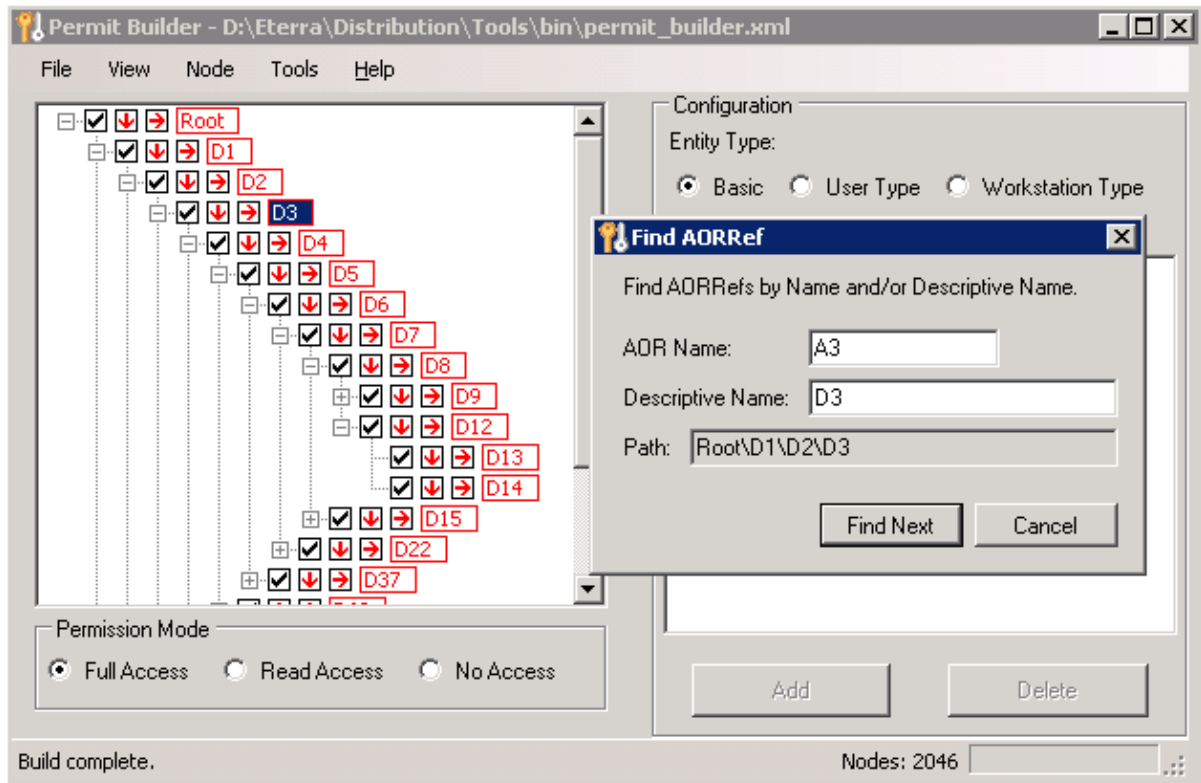
1. On the Node menu, click Find.

The FindAORRef window appears.



2. Search for a node using the AOR Name, the Descriptive Name, or both.

If a match is found, the tree view in the main window expands as needed to show the matching node. The matching node is also selected.

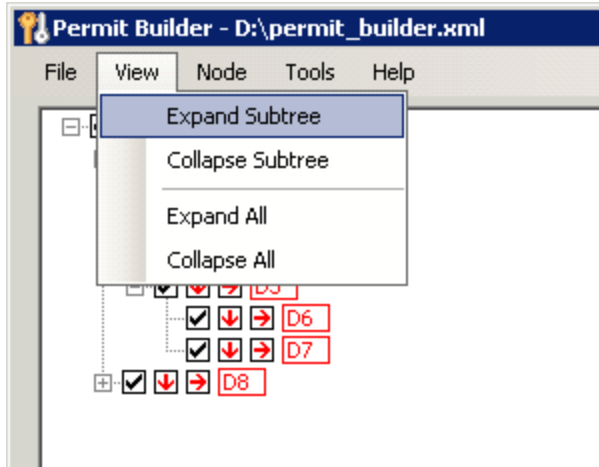


3. If multiple AORRefs match the search criteria, click the FindNext button to cycle through all of the matches.

3.1.8. Expanding and Collapsing a Subtree

To expand a subtree:

1. Select a node.
2. On the View menu, click Expand Subtree.



The selected subtree expands, along with all of its child nodes.

You can also expand all branches (not just the selected node) by clicking ExpandAll.

To collapse a subtree:

1. Select a node.
2. On the View menu, click Collapse Subtree.

You can also collapse all branches (not just the selected node) by clicking CollapseAll.

3.2. Operations with User Types and Workstation Types

This section describes how to add, edit, and delete User Types and Workstation Types.

3.2.1. Adding a User Type or Workstation Type

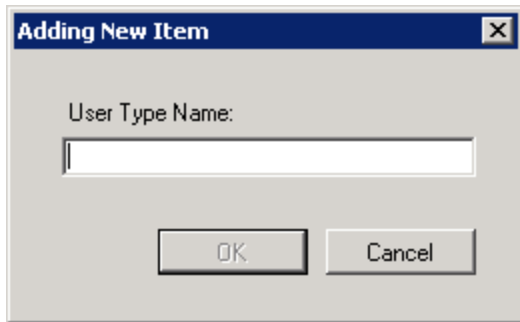
To add a new User Type or Workstation Type:

1. In the Configuration pane, select an entity type (User Type or Workstation Type).

The Add and Delete buttons become active.

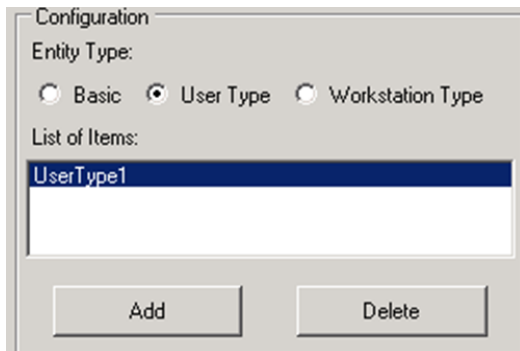
2. Click Add.

The Adding New Item dialog box appears.



3. Enter the name of the new entity type and click OK.

The new entity type appears in the list.



Note

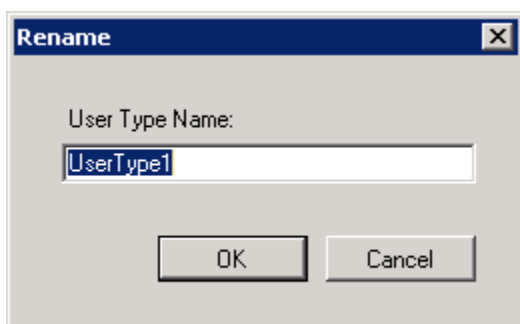
Initially, all new entity types inherit permissions from the Basic model. After that they can be configured as necessary.

3.2.2. Renaming a User Type or Workstation Type

To rename an entity type (User Type or Workstation Type):

1. In the Configuration pane, select an entity type.
2. Select an item and double-click it.

The Rename dialog box appears.



3. Enter the new name and click OK.

The item is renamed.

3.2.3. Deleting a User Type or Workstation Type

To delete an entity type (User Type or Workstation Type):

1. In the Configuration pane, select an entity type.

The Add and Delete buttons become active.

2. Select an item and click Delete.

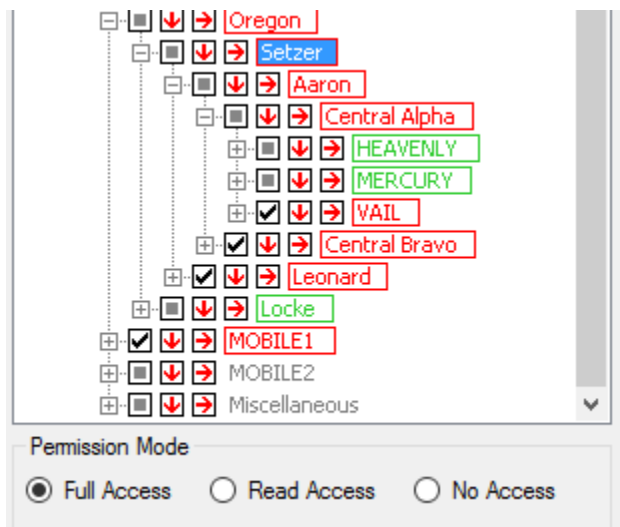
The item is removed from the model.

3.3. Modifying Access Levels for AORs

You can assign different access modes for AORs: full access, read-only access, and no access.

To modify an access level:

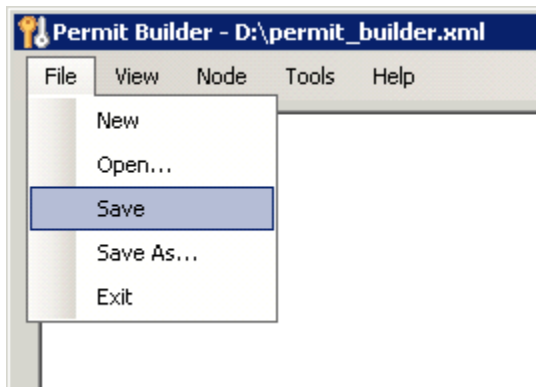
1. Select an entity type to be modified (Basic, User Type, or Workstation Type).
2. Select the Permission Mode. For more details about permission modes, see [Permission Modes](#).
3. Configure the access level using the right and down arrows near the AORs. For more details about permission mode controls, see [Permission Controls](#).



3.4. Saving and Importing the Model

To save the model:

1. On the File menu, click Save or SaveAs.



2. In the Save dialog box, name the file.

The default name for the model is "permit_builder.xml" but any file name can be used.

3. Click Save.

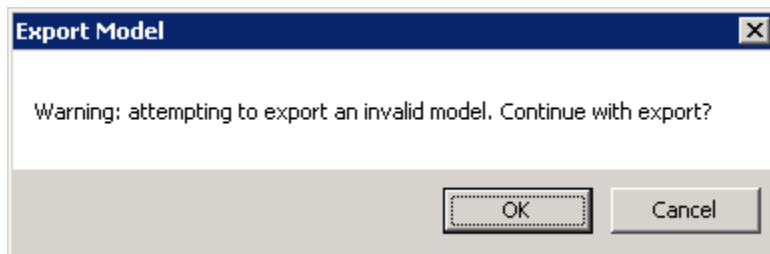
The AORRef hierarchy is saved in XML format.

To import a model:

1. On the File menu, click Open.
2. In the Open dialog box, browse for the desired XML file, and then click Open.

3.4.1. Validation Errors

If the model you are saving (that is, exporting) does not validate, the following message appears:



In this case, click OK to save the invalid model, or click Cancel to close the message without saving the invalid model.

For more information, see [Validating the AORRef Hierarchy](#).

4. Building the AORRef Hierarchy

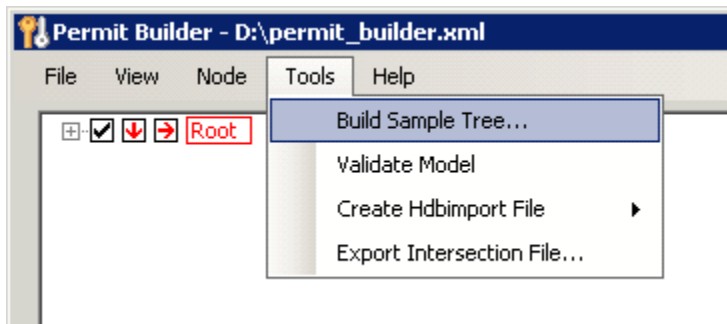
This chapter describes the process of building the AORRef hierarchy.

Permit Builder can build the basic AOR hierarchy and the hierarchy for User Types and Workstation Types.

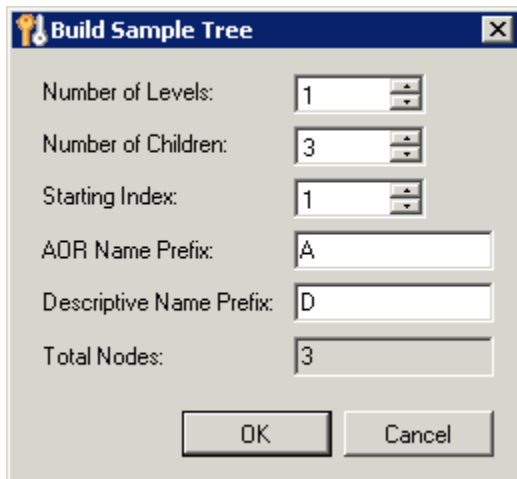
4.1. Building a Sample Tree

The quickest way to become familiar with the Permit Builder tool is to build a sample hierarchy.

1. On the Tools menu, click Build Sample Tree.



The Build Sample Tree dialog box appears.



2. Complete the dialog box and click OK.

The total number of nodes cannot be adjusted explicitly. This number is a calculated field, determined from the number of levels and the number of children per level. The maximum number of nodes is set at 100,000.

4.2. The AOR Reference XML Schema

This section describes the XML schema of the AOR References (AORRefs).

Consider the following simple model of the AORRef hierarchy:

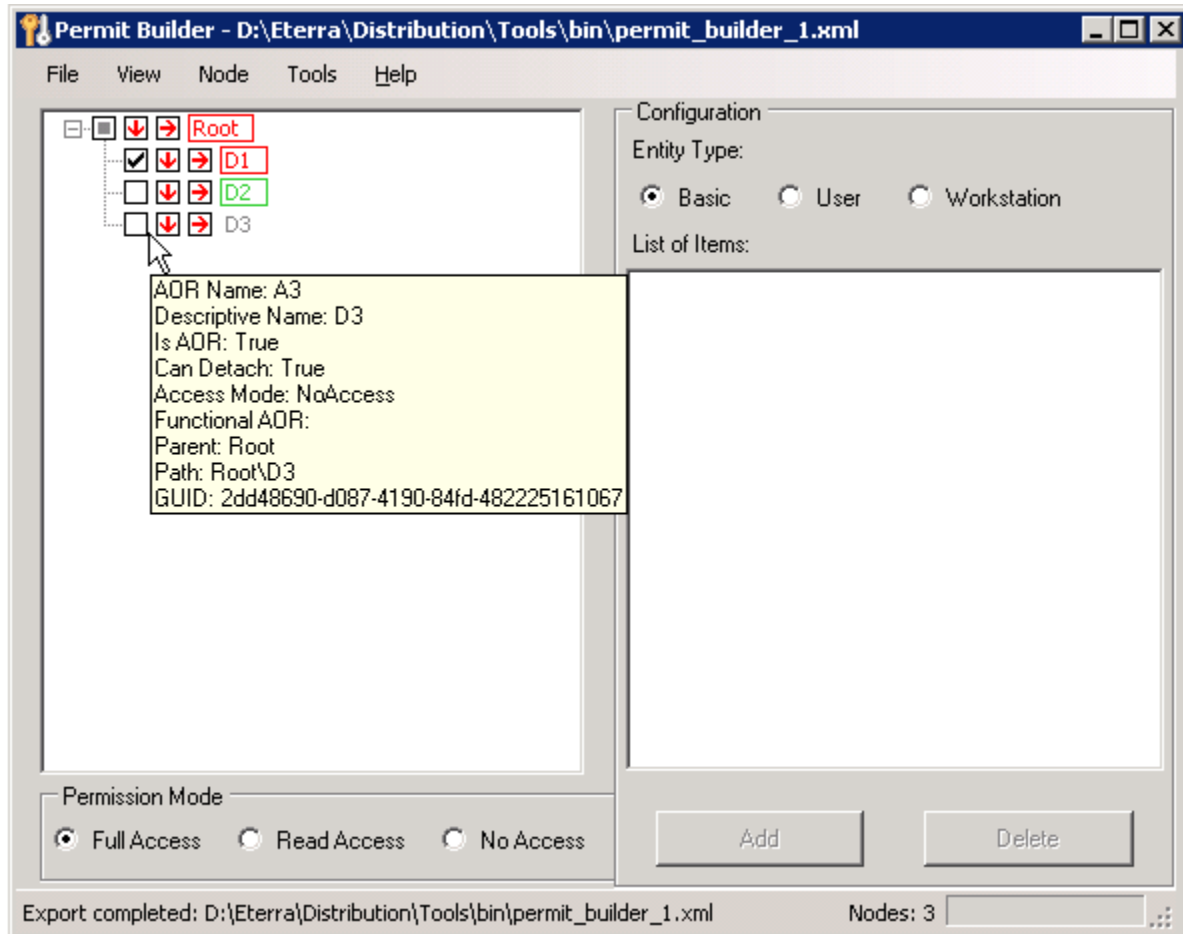


Figure 6. Simple Model of the AORRef Hierarchy

The following XML schema corresponds to this simple model; the below table describes the elements in the schema.

```
<?xml version="1.0" encoding="utf-8"?>
<AORConfig xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance">
  <AORs>
    <Name>AORs Tree</Name>
    <AORRef>
      <Children>
        <AORRef>
          <Children />
          <CheckedState>Checked</CheckedState>
          <IsExpanded>>false</IsExpanded>
          <CanDetach>>true</CanDetach>
        </AORRef>
      </Children>
    </AORRef>
  </AORs>
</AORConfig>
```

```

    <IsAOR>true</IsAOR>
    <DescriptiveName>D1</DescriptiveName>
    <FunctionalAOR />
    <NameAOR>D1</NameAOR>
    <GuidID>c2b7e1c2-5ccb-40b5-a825-cb1f0d8cd360</GuidID>
  </AORRef>
  <AORRef>
    <Children />
    <CheckedState>Unchecked</CheckedState>
    <IsExpanded>>false</IsExpanded>
    <CanDetach>true</CanDetach>
    <IsAOR>true</IsAOR>
    <DescriptiveName>D2</DescriptiveName>
    <FunctionalAOR />
    <NameAOR>D2</NameAOR>
    <GuidID>58870b2b-6c4d-4b93-b426-0d8c94464a61</GuidID>
  </AORRef>
  <AORRef>
    <Children />
    <CheckedState>Unchecked</CheckedState>
    <IsExpanded>>false</IsExpanded>
    <CanDetach>true</CanDetach>
    <IsAOR>true</IsAOR>
    <DescriptiveName>D3</DescriptiveName>
    <FunctionalAOR />
    <NameAOR>D3</NameAOR>
    <GuidID>2dd48690-d087-4190-84fd-482225161067</GuidID>
  </AORRef>
</Children>
<CheckedState>Grayed</CheckedState>
<IsExpanded>true</IsExpanded>
<CanDetach>true</CanDetach>
<IsAOR>false</IsAOR>
<DescriptiveName>Root</DescriptiveName>
<FunctionalAOR />
<NameAOR>ROOT</NameAOR>
<GuidID>1c5dc0fb-9334-4560-b5ef-e35d07e152b7</GuidID>
</AORRef>
</AORs>
<DefaultPermissions>
  <Name>Default Permissions</Name>
  <AreaModes>
    <AreaMode>
      <Guid>1c5dc0fb-9334-4560-b5ef-e35d07e152b7</Guid>
      <Mode>FullAccess</Mode>
    </AreaMode>
    <AreaMode>
      <Guid>c2b7e1c2-5ccb-40b5-a825-cb1f0d8cd360</Guid>
      <Mode>FullAccess</Mode>
    </AreaMode>
    <AreaMode>
      <Guid>58870b2b-6c4d-4b93-b426-0d8c94464a61</Guid>
      <Mode>ReadAccess</Mode>
    </AreaMode>
  </AreaModes>

```



```

</AreaMode>
<AreaMode>
  <Guid>2dd48690-d087-4190-84fd-482225161067</Guid>
  <Mode>NoAccess</Mode>
</AreaMode>
<AreaMode>
  <Guid>1c5dc0fb-9334-4560-b5ef-e35d07e152b7</Guid>
  <Mode>FullAccess</Mode>
</AreaMode>
<AreaMode>
  <Guid>c2b7e1c2-5ccb-40b5-a825-cb1f0d8cd360</Guid>
  <Mode>FullAccess</Mode>
</AreaMode>
<AreaMode>
  <Guid>58870b2b-6c4d-4b93-b426-0d8c94464a61</Guid>
  <Mode>FullAccess</Mode>
</AreaMode>
<AreaMode>
  <Guid>2dd48690-d087-4190-84fd-482225161067</Guid>
  <Mode>FullAccess</Mode>
</AreaMode>
</AreaModes>
</DefaultPermissions>
<Users />
<Workstations />
</AORConfig>

```

Table 4. Description of the XML Schema

Element	Description
AORs	Base model AORRef records.
Name	AOR name (maximum 8 characters, no embedded spaces).
AORRef	AORRef records.
Children	AORRef child records.
CheckedState	AOR access state (Checked, Unchecked, or Grayed).
IsExpanded	Boolean data type: • True if the AORRef is expanded in the default state. • False if the AORRef is collapsed in the default state.
CanDetach	Boolean data type: • True if the AORRef can be detached. • False if the AORRef cannot be detached.
IsAOR	Boolean data type: • True if an AREA in Permit. • False if not an AREA in Permit.
DescriptiveName	Descriptive name.

Element	Description
FunctionalAOR	Functional AOR name (maximum 8 characters, no embedded spaces). This field is optional.
NameAOR	AOR name (maximum 8 characters, no embedded spaces).
GuidID	Globally unique identifier (GUID).
DefaultPermissions	Area records and their permission modes for an AOR model.
AreaModes	AreaModes records.
AreaMode	Records for a particular area mode.
Guid	GUID of the area mode.
Mode	Permission mode of the area mode (FullAccess, ReadAccess, or NoAccess).
Users	User type AORRef records (if user types are defined).
Workstations	Workstation type AORRef records (if workstation types are defined).

4.3. Building a Basic Hierarchy

The Basic hierarchical model should contain all Areas of Responsibility.

To build a Basic hierarchy:

1. On the File menu, click New.
2. Add AOR nodes in the tree structure. For more information, see [Operations with Nodes](#).
3. Configure the access level for the AOR nodes, as described in [Modifying Access Levels for AORs](#).
4. Save the hierarchy.

4.4. Building the Hierarchy via an External Script

The AORRef hierarchy can be built manually, using the basic operations outlined in the previous chapter. However, for large databases, it may be more efficient to write an external script to generate the AORRefXML automatically. For a description of the XML schema, see [The AOR Reference XML Schema](#).

After you build the hierarchy via an external script, import the XML, as described in [Saving and Importing the Model](#).

Finally, validate the model, as described in [Validating the AORRef Hierarchy](#).

4.5. Building a User Type and Workstation Type Hierarchy

All User Types and Workstation Types have their own hierarchical models. When a new User or Workstation Type is created, it inherits the Basic model, which is used as a default model for new types.

To build a User Type and Workstation Type hierarchy:

1. Build a Basic hierarchy.
2. Add new User Types and Workstation Types.
3. Modify the AORs tree and configure the access levels for each User and Workstation Type.
4. Save the hierarchy.

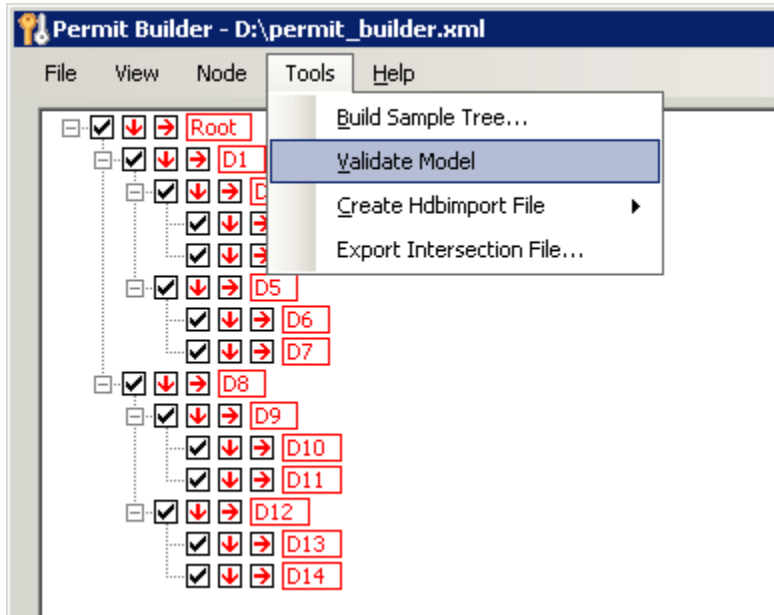
5. Validating the AORRef Hierarchy

This chapter describes the process of validating the AORRef hierarchy.

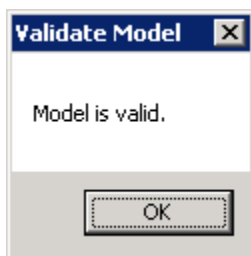
5.1. Validating the Model

To validate the model:

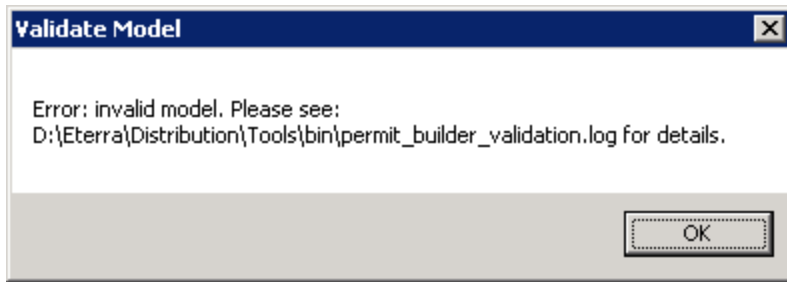
1. Open the Tools menu and select Validate Model.



If the model validates successfully, the following message appears:



If the model fails validation, the following message appears:



2. To view the validation errors, open the permit_builder_validation.log file in the program directory.

5.2. Defining the Validation Criteria

This section describes different validation criteria.

5.2.1. Circular References

A "circular reference" is an AORRef that exists as a descendant of itself.

If circular references exist in the hierarchy, the model does not validate.

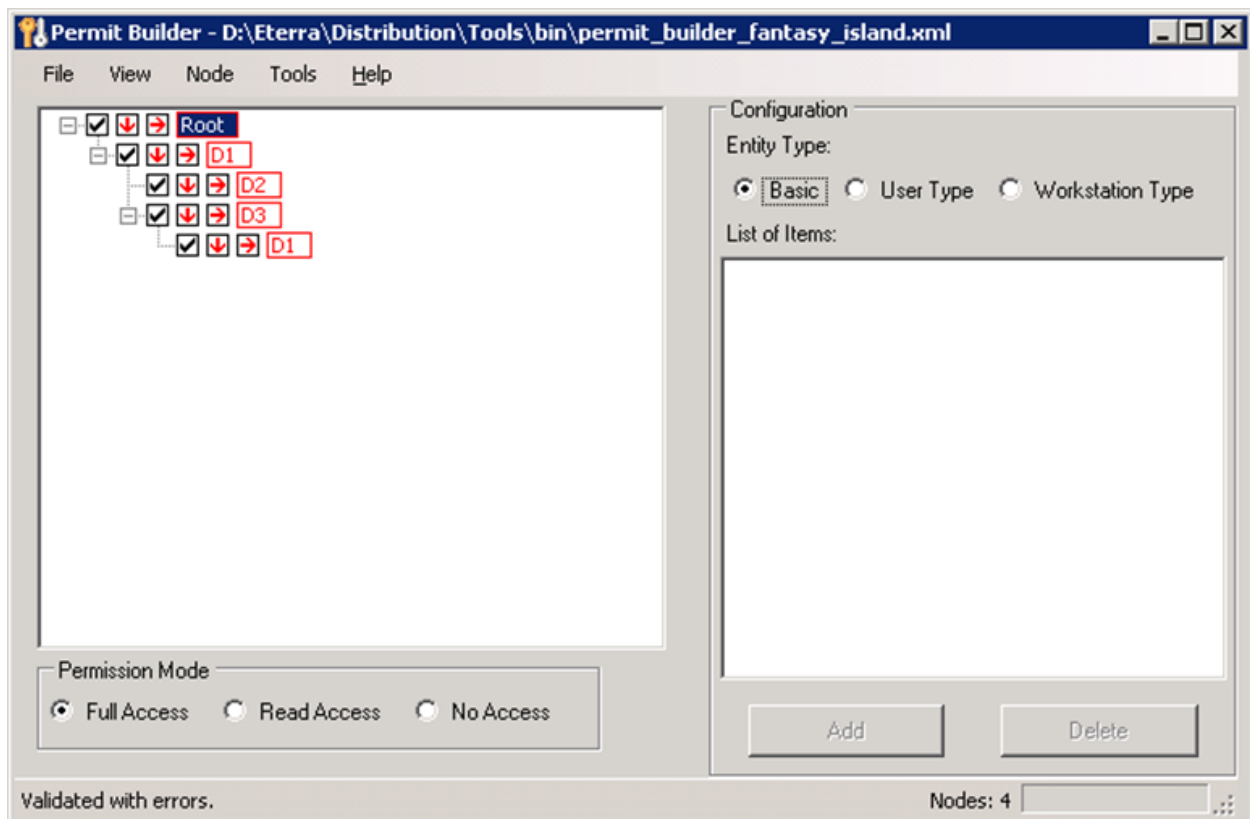


Figure 7. Example of a Circular Reference

This example shows how circular reference errors appear in the validation log file:

```
10/22/2015 8:15:53 AM: Circular reference [1]: Root\D1
10/22/2015 8:15:53 AM: Circular reference [1]: Root\D1\D3\D1
```

5.2.2. Invalid AOR Name

Since the AOR Name corresponds to an AREA record in PERMIT, the AORName must comply with Hdb standards. In particular, the following must be true:

- The AOR Name cannot contain more than eight characters.
- The AOR Name cannot contain embedded spaces.

This example shows how invalid AOR Name errors appear in the validation log file:

```
10/22/2015 2:20:19 PM: AOR Name [A123456789] has more than 8 characters for Descriptive Name: D280
10/22/2015 2:20:19 PM: AOR Name [A 15] contains embedded spaces for Descriptive Name: D15
```

5.2.3. Inconsistency between the AOR Name and IsAOR Attributes

If the Boolean attribute IsAOR is True, then the corresponding AORName cannot be blank.

The following is an example of this error as it appears in the validation log file:

```
10/22/2015 2:20:19 PM: AOR Name is blank and IsAOR is TRUE for Descriptive Name: D10
```

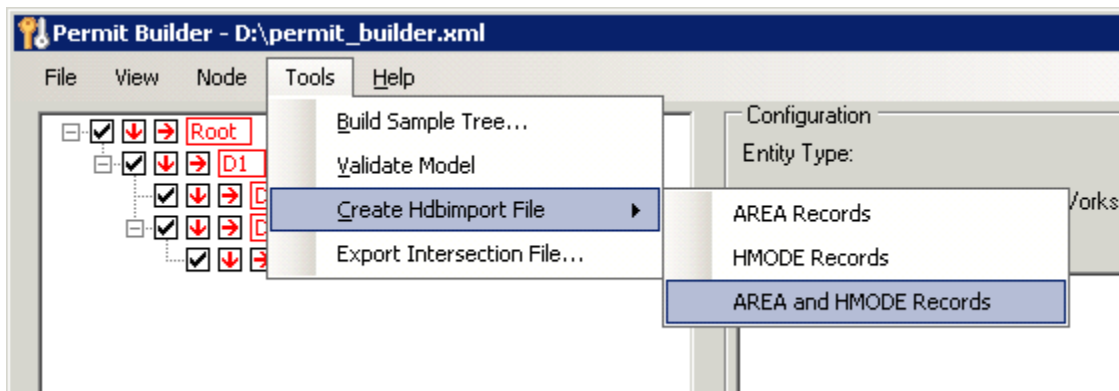
6. Creating Hdbimport Files

This chapter describes the process of exporting the model as an hdbimport file.

6.1. Exporting the Model in Hdbimport Format

To export the model in hdbimport format:

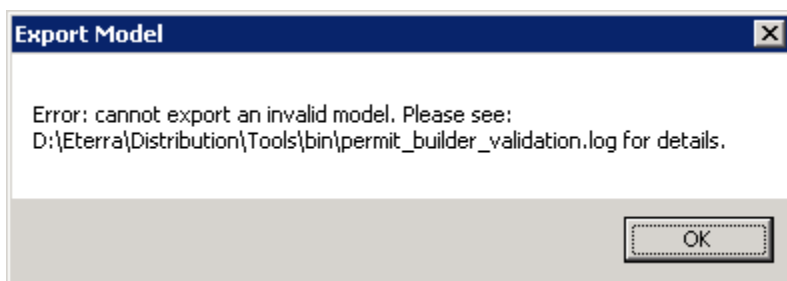
1. On the Tools menu, click Create Hdbimport File, and then select one of the following options:
 - **AREA Records:** Export the AREA records to an hdbimport file called `hdbimport_area.txt`.
 - **HMODE Records:** Export the HMODE records to an hdbimport file called `hdbimport_hmode.txt`.
 - **AREA and HMODE Records:** Export the AREA and HMODE records to an hdbimport file called `hdbimport.txt`.



The hdbimport file `hdbimport.txt` is created in the parent folder of the Permit Builder executable (by default, `d:\Eterra\Distribution\Tools\bin`).

2. If the model is valid, an hdbimport file is created in the program directory.

If the model is invalid, the following dialog box appears:



To reduce the chances of corrupting the PERMIT database, PermitBuilder prevents exporting an invalid model. Correct the validation errors and export the model again.

6.2. Example of the Hdbimport File

The following is the hdbimport file that corresponds to the simple model from Chapter1, using the AREA and HMODE records option in PermitBuilder.

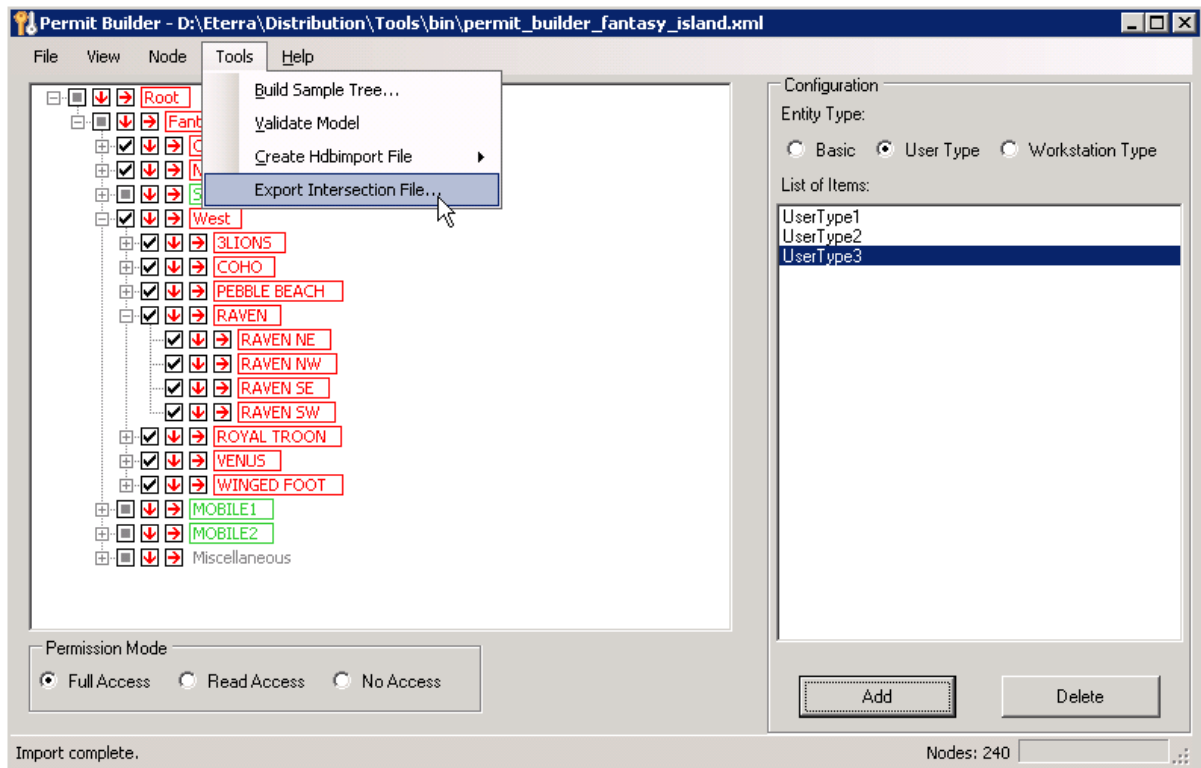
```
# =====
# This hdbimport script was created by Permit Builder on 7/1/2015 2:27:19 PM.
#
# To import, run the following commands from a command prompt.
#
# > context permit habitat
# > hdbimport -d permitnv -s hdbimport.txt -ignore_indirect_msg
#
# =====
# Add HMODE records.
# =====
#record
HMODE,%SUBSCRIPT,ID_HMODE,AREAIID_HMODE,DESC_HMODE,AREAFUN_HMODE,ISAOR_HMODE,NOACCESS_HMODE,ONLY_HMODE
,CANDTCH_HMODE,INTERNAL_HMODE,DO__HMODE_HMODE,UP__HMODE_HMODE,F__HMODE_HMODE,A__HMODE_HMODE,B__HMODE_H
MODE
HMODE,1,"b1fb5a1c-4407-416c-9f72-296ee919d60c","ROOT","Root","",F,F,F,T,F,2,0,0,0,0
HMODE,2,"012a40ee-c3cc-4a6f-bdae-eda620447572","A1","D1","",F,T,F,T,F,4,1,0,0,0
HMODE,3,"434fbb94-6998-4fbd-a50a-705efcf14761","A2","D2","",T,F,F,T,F,0,2,0,0,0
HMODE,4,"6f07c079-ce0c-4a4e-b2f6-8a252d87ce79","A1","D1 (INTERNAL)",",",T,F,F,T,T,0,2,3,0,0
#
# =====
# Do not insert duplicate records.
# =====
#
#insertnodup
#
# =====
# Add AREA records.
# =====
#record AREA,ID_AREA,DESCRIP1_AREA,CREAD_AREA,CWRITE_AREA,CEXEC_AREA,CAUD_AREA
AREA,'A1','D1',F,F,F,F
AREA,'A2','D2',F,F,F,F
# =====
# Total Areas = 2
# =====
```

6.3. Exporting an Intersection Model File

The intersection model file contains pairs of users and workstations (User;Workstation) and their permissions (FullAccess, ReadAccess, or NoAccess) for a given area. The intersection model file is created based on the User Type and Workstation Type hierarchy.

To export an intersection model file:

- On the Tools menu, click Export Intersection File.



The intersection model file Intersections.txt is created in the parent folder of the Permit Builder executable (by default, d:\Eterra\Distribution\Tools\bin).

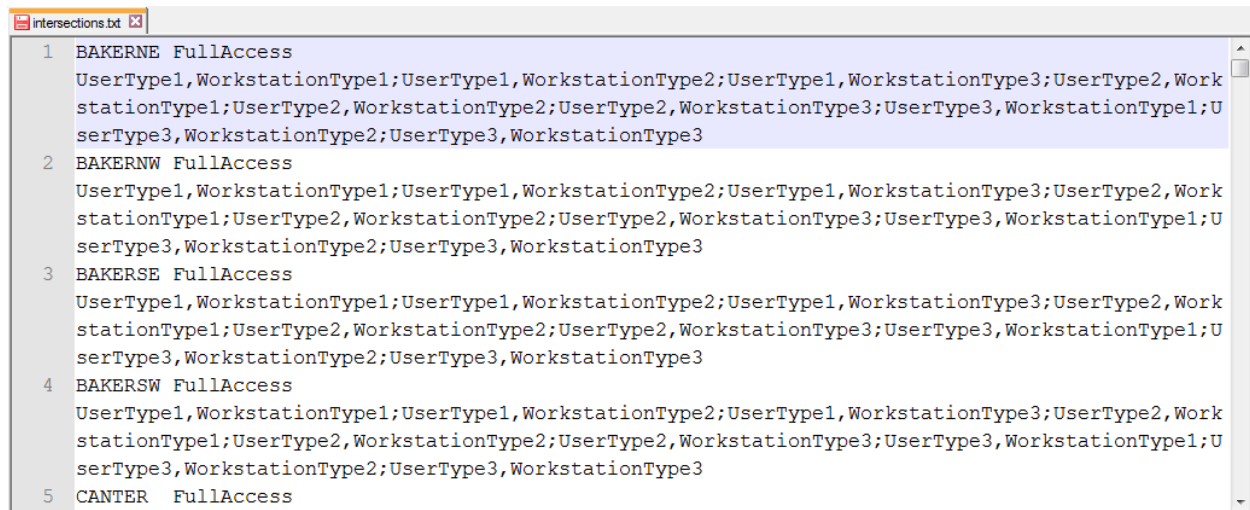


Figure 8. Example of an Intersection Model File

6.4. Exporting Models from the Command Line

The Permit Builder's command-line interface supports the following options.

/CONFIG [PATH TO FILE]

Specifies the path to the Permission Management plug-in configuration file. If the Permit Builder is run with this option, it will have the same colors and settings as the Permission Management plug-in.

/INPUT [PATH TO FILE]

Specifies the path to the AORRef hierarchy model.

/OUTPUTPATH [PATH TO FOLDER]

Specifies the path to the output folder for hdbimport and intersection files. If this option is not specified, the default location is used (D:\Eterra\Distribution\Tools\bin).

/OUTPUT [NAME OF THE OUTPUT HDBIMPORT FILE]

Specifies the name of the output hdbimport file. If this option is not specified, the default file name is used (hdbimport.txt).

/INTERSECTION [PATH TO THE OUTPUT INTERSECTION FILE]

Specifies the path to the output intersection model file. If this option is not specified, the file is exported to the OUTPUTPATH folder, if specified, and the default file name is used (Intersections.txt).

/RECORD [FILE TYPE]

Specifies the type of the output file to be created.

- **HMODE:** Use this argument to create an hdbimport file with the HMODE records (hdbimport_hmode.txt).
- **AREA:** Use this argument to create an hdbimport file with the AREA records (hdbimport_area.txt).
- **UWMODE:** Use this argument to create an intersection model file (Intersections.txt).
- **ALL:** Use this argument to create an hdbimport file with both the AREA and HMODE records (hdbimport.txt) and an intersection model file (Intersections.txt).

/VERBOSE

Add this option to an import or export command to show the command processing status in the command line.

/H

Shows the help output that lists all of the tool's command-line parameters.

Exporting an hdbimport file

You can export an hdbimport file using the following command:

```
> PermitBuilder.exe /INPUT "path to the XML model input file" /OUTPUTPATH "path to the output folder" /OUTPUT "name of the output hdbimport file" /RECORD "output file type"
```

For example:

```
> D:\Eterra\Distribution\Tools\bin\PermitBuilder.exe /INPUT D:\Eterra \Distribution
\Tools\bin\permit_builder_fantasy_island.xml /OUTPUTPATH D:\Eterra \Distribution \Tools \bin /OUTPUT
hdbimport_hmode.txt /RECORD HMODE
```

Exporting an intersection model file

You can export an intersection model file using the following command:

```
> PermitBuilder.exe /INPUT "path to the XML model input file" /INTERSECTION "path to the output
intersection model file" /RECORD UWMODE
```

For example:

```
> D:\Eterra\Distribution\Tools\bin\PermitBuilder.exe /INPUT D:\Eterra \Distribution
\Tools\bin\permit_builder_fantasy_island.xml /INTERSECTION D:\Eterra \Distribution \Tools
\bin\Intersections.txt /RECORD UWMODE
```

Exporting all files

You can export both an hdbimport file and an intersection model file using the following command:

```
> PermitBuilder.exe /INPUT "path to the XML model input file" /OUTPUTPATH "path to the output folder"
/OUTPUT /RECORD "ALL"
```

For example:

```
> D:\Eterra\Distribution\Tools\bin\PermitBuilder.exe /INPUT D:\Eterra \Distribution
\Tools\bin\permit_builder_fantasy_island.xml /OUTPUTPATH D:\Eterra \Distribution \Tools \bin /RECORD
ALL
```

7. Modifying the Permit Model with a Script

The `pb.pl` script uses the Permit Builder application to model not only AREA and HMODE records, but also the initial permissions for user types and workstation types in the Permit model. The modified permission records support authorization under the Intersection model, which is the intersection of the user-centric and the workstation-centric models.

The script directly modifies the PERMITNV database. It works as follows:

1. Creates a savecase of the existing PERMITNV database to enable the backup.
2. Reads the input file with the intersection model. The intersection model contains combinations of user types and workstation types for particular AORs, including permission modes.
3. Modifies the AREA, HMODE, MODE, USRTYP, UMODE, WSTYP, and WMODE tables in the PERMITNV database using the data from the input file.
4. Creates two-dimensional records (MODE/AREA) with access type settings (Read, Write, Execute, and NoAccess) for both modes and areas.

The script creates modes for intersections (combinations) between user types and workstation types that are included in the input file. The number of modes is $N \times M$, where N is the number of user types and M is the number of workstation types. For all generated modes, the specified access levels for the specified areas are set.

Note

The default size of the Permit database may not be large enough to store all of the generated modes. Therefore, you may need to expand the database. For more information, refer to the Platform *Database Resizing Utility User Guide*.

5. Runs the validation of the PERMITNV database.

If any errors occur during the validation, the PERMITNV database is restored from the savecase.

To run the script, the following parameter is mandatory:

```
-i <input file path>
```

The additional parameters are:

```
-a <db application>
-f <db family>
-d <db name>
-m <default mode for all user and workstation types>
```

If not specified, the following default values are used:

```
-a PERMIT
-f EMS
-d PERMITNV
```

7.1. Running the Script

Before running the script, ensure that the Habitat Server (HDBSERVER) is running.

In addition, to enable the PERMITNV database validation, start Permit Server (PERMSRV) and RUSER Server (RUSERSERVER). If either of them is not running, the validation is skipped.

1. Create an input file for the script manually or using the Permit Builder, as described in [Exporting an Intersection Model File](#).

The input file specifies the Area access parameters to be modified by the script. It contains the following data:

- **Area:** An operational environment that can be given permissions.

Note

The areas must already be defined in the Permit model.

- **Access Level:** Access level for the area (NoAccess, ReadAccess, or FullAccess).
- **User Type:** The User Type that is granted the specified permissions for the area.

If a particular User Type was not defined in the Permit model before running the script, this User Type will be added to the model after the script is executed.

- **Workstation Type:** Workstation Type that is granted the specified permissions for the area.

If a particular Workstation Type was not defined in the Permit model before running the script, this Workstation Type will be added to the model after the script is executed.

The input file format is the following:

```
<Area1> <Access Level> <User Type>,<Workstation Type>
```

or:

```
<Area2> <Access Level> <User Type1>,<Workstation Type1>;<User Type2>,<Workstation Type2>
```

Items must be delimited as follows:

- The area, access level, and user type entries must be separated with tabs.
- The user type and workstation type entries must be separated with commas.
- The user type and workstation type pairs must be separated with semicolons.
- The rows must be separated with paragraphs.

For example:

```
PLOUTAGE FullAccess UTDMSALL,GUEST
CONFOPER FullAccess UTDMSALL,GUEST
```

2. Change the Workstation Type models to Intersection for all Workstation Types that already exist in the Permit model and that are included in the input file (refer to [Changing the Workstation Type Model](#)).
3. From the Habitat command prompt, go to the script folder and run the script. For example:

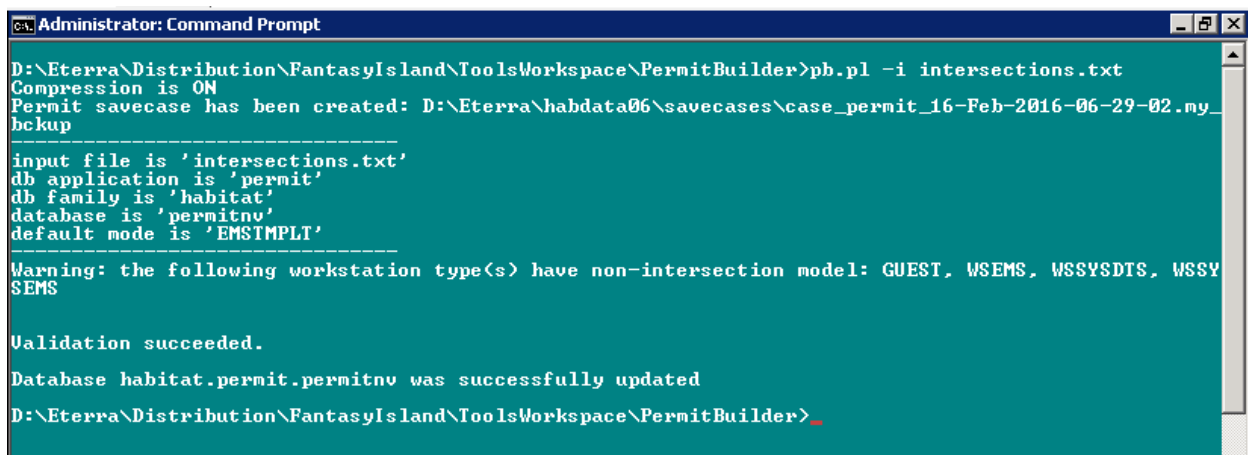
```
D:\Eterra\Distribution\FantasyIsland\ToolsWorkspace\PermitBuilder>pb.pl -i intersections.txt
```

The script creates a savecase before updating the Permit model. After creating new AORs and modes, the script calls the Habitat "validatedb" command through the RUSER API.

4. Review the command prompt and validation log for errors, if any.

If PERMLOG contains new errors or fatals after the script runs, the script automatically restores the Permit savecase from backup.

5. After the script is successfully run, restart the Habitat server and start the ADMS servers to apply any changes.



```
Administrator: Command Prompt
D:\Eterra\Distribution\FantasyIsland\ToolsWorkspace\PermitBuilder>pb.pl -i intersections.txt
Compression is ON
Permit savecase has been created: D:\Eterra\habdata06\savecases\case_permit_16-Feb-2016-06-29-02.ny_
backup
-----
input file is 'intersections.txt'
db application is 'permit'
db family is 'habitat'
database is 'permitnv'
default mode is 'EMSTPLT'
-----
Warning: the following workstation type(s) have non-intersection model: GUEST, WSEMS, WSSVSDIS, WSSV
SEMS

Validation succeeded.
Database habitat.permit.permitnv was successfully updated
D:\Eterra\Distribution\FantasyIsland\ToolsWorkspace\PermitBuilder>
```

Figure 9. Example of a Command-Line Output for Running the Script

7.2. Changing the Workstation Type Model

The Intersection model must be applied to all workstation types that are included in the script. Other workstation type models (User-Centric and Workstation-Centric) will not be processed correctly for the modified areas.

To change the workstation type model:

1. Go to the Mode Directory analyst display (Analyst Displays menu> Workstation Modeling> Workstation Type Modeling).
2. Change all User-Centric and Workstation-Centric models to Intersection (right-click the mode

and select Intersection).

Caution

Do not modify the workstation types not included in the script.

3. Click Validate Model.
4. Click Apply Model.

7.3. Verifying Script Results

1. Open the Permit display (HABITAT menu> Permit> Active Sessions).
2. Go to the User Type Modeling analyst display (Analyst Displays menu> User Modeling> User Type Modeling).
3. Verify that the new user types were added by the script. Note the indexes of the new user types (for example, 18 and 19).

User Type modeling		Validate Model	Apply Model	Show Modes
User Type 				
 	1	ERRMOD		
 	2	ADMIN		
 	3	USRGEN		
 	4	USREMS		
 	5	USRDTS		
 	6	USRNET		
 	7	INSTRUCT		
 	8	TRAINEE		
 	9	USRNORTH		
 	10	USREAST		
 	11	USRSOUTH		
 	12	USRECAR		
 	13	UTDMSALL		
 	14	UTDMSCTL		
 	15	UTDMSNEA		
 	16	UTDMSSTH		
 	17	UTDMSWST		
 	18	USER2		
 	19	USER1		

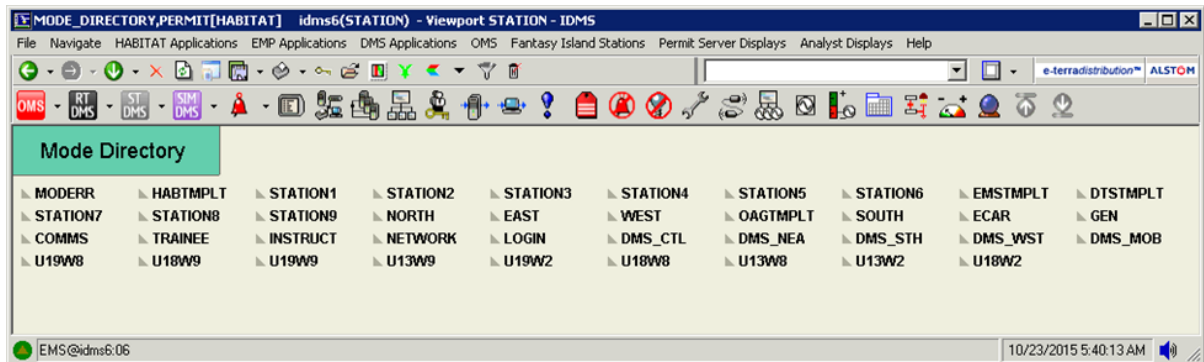
- Go to the Workstation Type Modeling analyst display (Analyst Displays menu> Workstation Modeling> Workstation Type Modeling).
- Verify that the new workstation types were added by the script. Note the indexes of the new workstation types (for example, 8 and 9).

Workstation Type Modeling Validate Model Apply Model [Show Modes](#)

Workstation Type ▾

1	MODERR	>>>	Intersection
2	GUEST	>>>	Intersection
3	WSSYSEMS	>>>	WS-centric
4	WSSYSDTS	>>>	WS-centric
5	WSGEN	>>>	Intersection
6	WSEMS	>>>	User-centric
7	WSNET	>>>	Intersection
8	WRKST1	>>>	Intersection
9	WRKST2	>>>	Intersection

- Go to the Mode Directory analyst display (Analyst Displays menu> ModeModeling> Mode Directory).
- Verify that the new modes for the user types and the workstation types were added by the script.



The new intersection modes are composed of the indexes of their user types and workstation types. For example, the U19W8 mode is an intersection of the user type with index = 18 (U18), and the workstation type with index = 8 (W8).

- Right-click a new mode and click Mode Areas.
- Navigate to the added AREA (for example, PLOUTAGE) and verify that all Read, Write, and Execute check boxes are set, which means the FullAccess permission mode.

OVERNCD	☐ Read	☐ Write	☐ Execute	☐ Audible
CONFOPER	☐ Read	☐ Write	☐ Execute	☐ Audible
TFTEST	☐ Read	☐ Write	☐ Execute	☐ Audible
PLOUTAGE	☒ Read	☒ Write	☒ Execute	☐ Audible

DMSALL@idms6:06

10. Open the Permit main display (HABITAT menu> Permit> Active Sessions).
11. Open the Active Modes tab and make sure that a new mode that was initialized for the current user and the workstation is active.

Permit				
Session Information -- Active Modes -- Available Modes			Number of Sessions: 1	
Initial Login Time	User ID	Workstation	Permit Console	Active Modes
23-Oct-2015 06:25:48	DMSALL	IDMS6	GUEST00	EMSTMPLT (D) U13W2 (I)

12. Check the Permit server log (HABITAT menu> Permit> Permit Server Log) for warning and errors.